

AI Whiteboard



WPI

A Progress Report Submitted to:

Professor Ben Ngan

Authors:

Edward Fantasia, Quinn Ferguson, Nicholas Heineman,
Anthony Virone and Atharva Tiwari

Abstract:

The rise of virtual environments post-COVID-19 has highlighted the need for more intuitive and effective tools for communication, particularly in virtual meetings and online learning. Conventional methods of interaction, such as drawing with a mouse, are limited in both precision and user engagement. We explore several state-of-the-art (SOTA) hand gesture recognition methods, including YOLOv3 with DarkNet-53, Faster R-CNN, mIV3Net, EfficientNet, and ConvNeXt, evaluating their suitability for real-time, low-resource environments. Each model's strengths and challenges are discussed, and preliminary results guide the selection of the optimal approach for integrating hand gesture recognition into an AI-driven whiteboard application. Through testing and comparison, we aim to create a system that enhances user engagement, reduces computational overhead, and provides an accessible, innovative solution for digital collaboration. This paper proposes an AI-powered whiteboard system, leveraging Google's Media Pipe Hands to enable hand gesture-based drawing using simple cameras. This solution aims to overcome existing limitations of traditional whiteboard tools, fostering creativity and interactivity in online platforms like Zoom and Microsoft Teams. The system has significant applications across education, enabling more dynamic and collaborative learning experiences, especially for students with disabilities. We also discuss potential future work, including further optimizations in gesture recognition accuracy, real-time performance, and exploring additional applications beyond digital collaboration tools.

Table of Contents

Table of Contents.....	3
Project Team.....	4
Introduction.....	5
SOTA Literature Review.....	7
YOLOv3 and DarkNet-53.....	7
EfficientNet.....	8
Inception V3.....	9
ConvNeXt.....	11
MediaPipe Hands.....	13
Proposed High-level Solution and Process.....	15
EfficientNet.....	15
Inception V3.....	15
ConvNeXt.....	16
MediaPipe Hands.....	16
Preliminary Experimental Results and Discussions.....	18
YOLOV3.....	18
EfficientNet.....	18
Inception V3.....	18
ConvNeXt.....	19
MediaPipe Hands.....	20
● Selected model: MediaPipe Hands.....	20
Lessons Learned.....	20
Future Work.....	21
Conclusions.....	21
References.....	21
Appendixes.....	24
Appendix I.....	24
Appendix II.....	25

Project Team

Edward Fantasia
MS Computer Science
Georgetown, MA



Quinn Ferguson
MS Robotics Engineering
Charlottesville, VA



Nicholas Heineman
MS Computer Science
Framingham, MA



Anthony Virone
MS Robotic Engineering
Toms River, NJ



Atharva Tiwari
MS Computer Science
Farmington, CT



Introduction

After the COVID-19 pandemic, our world has become increasingly virtual, with students and companies relying more on virtual meeting technologies than ever before. One field that we believe technology is lacking in is an easy way to express ideas via drawing during recordings and meetings. Conventional drawing with a mouse is difficult and usually does not produce great results. To fix this, we believe creating an AI whiteboard, where the user draws on the screen with their fingers and hand gestures, will be easier and more effective than a conventional mouse.

While virtual meeting platforms like Zoom, Webex and Microsoft Teams have made important strides in enhancing collaboration, they often fall short in enabling creativity and “thinking out problems.” Existing digital whiteboards typically require hardware solutions, such as a pen, touch screen monitor or janky mouse input that is often not legible. AI-powered whiteboards that are driven by CV technologies can overcome these limitations by interpreting hand movements and symbols using simple cameras that nearly everyone has either on their phone or personal computer. Furthermore, advancements in deep learning and CV have made it possible to enable these systems to be used by individuals that are disabled or are unable to use a mouse or pen to effectively engage.

A CV supported whiteboard system has broad applications across industries and educational environments. A recent Washington Post article summarizes many students’ and educators’ experience by citing it as “teaching into a void (Nguyen, 2021).” One of the reasons is because you cannot feed off of the energy of the class, their questions and their problem solving. On the other hand, from the student perspective, in the 2022-23 academic year about 53% of U.S students were enrolled in at least one online course (Hamilton and Beagle, 2023). While many students enjoy the flexibility of online classes, many complain about the lack of engagement, often stating that lectures are not collaborative as they are in-person. This is an issue that a well engineered CV Whiteboard can resolve, it can serve as a tool for interactive teaching, enabling them to annotate slides and solve questions for students without displaying a whiteboard. On the other hand, it can allow students to be more engaged in the lecture, participate and bring the spontaneity and fluidity of an in-person class to an online setting.

In sum, a well-engineered AI-powered whiteboard tool can not only enable educators to better improve the quality of online courses but also allows students to express themselves in an environment that helps their learning as well as their classmates. This tool can also help students that are disabled or cannot use traditional means of communicating in an online setting to express themselves and increase their quality of education and social interaction.

One state-of-the-art (SOTA) method that may be used for AI hand gesture recognition is the YOLOv3 model and DarkNet-53 convolutional neural network. Using the method described in *Real-Time Hand Gesture Recognition Based on Deep Learning YOLOv3 Model*, the model can achieve high accuracy in complex environments and with low-resolution images. If we decide to use the YOLO approach, we will have to conduct tests to determine if YOLOv3 and DarkNet-53 is accurate enough for our needs while having a fast detection speed and low training time. It is also important to make sure our computers are able to run programs using this method and that this method does not require excessive processing power. Specifics regarding what YOLOv3 and DarkNet-53 are will be described in the *SOTA Literature Review* section of this paper (Mujahid et al., 2021).

R-CNN is a state-of-the-art method that has been used for detecting objects in images and videos, and as described in *Hier R-CNN: Instance-Level Human Parts Detection and A New Benchmark*, can be modified to work with identifying hands. It relies on an algorithm called Selective Search that narrows the picture down to certain regions and locations and refines from there. Additionally, there is Faster R-CNN that refined the process to make it much faster. An advantage of this system is eliminating the need for computationally expensive analysis of each photo, instead relying on a pre-trained system. Additionally, it can track multiple objects at once which would be useful for just hand motions as well as other body motions. A disadvantage is that it is still somewhat slow, and may not be powerful enough for regular video footage. If we decide to use this, the algorithm may have to be limited and somewhat selective for what we want to do. (L. Yang et al., 2021)

Another interesting SOTA method that can be used for AI hand recognition was developed and researched by Karsh et al.(2023) and is called mIV3Net. First, it is important to address what this deep learning model is, it is a

3D convolutional neural network (CNN) that focuses on the “spatial-temporal relationships” that exist in image and video data. This method like the others discussed above is designed for hand gesture recognition (HGR) and these researchers take the base inception V3 network and architecture, improve it to reduce computational overhead as well as fine-tune the model so that even in complex backgrounds the features (gestures) are extracted. When training their model, they adopted a technique called Transfer Learning, this was able to improve the new network’s performance and reduce the computational and time requirements needed to normally train the model. Finally, when testing their model against 5 publicly available datasets their accuracy values were higher than that of other SOTA HGR systems that exist (Karsh et al., 2023).

Another interesting SOTA method that can be used for AI hand recognition is ConvNeXt. As stated in the paper *A ConvNet for the 2020s*, ConvNeXt is a convolutional neural network, which is a neural network that learns features through filters. ConvNeXt is different in that it is a “family of pure convolutional neural network models” that are competitive with the accuracy and scalability of Transformers, a type of neural network that uses attention to learn context to then generate data (Liu et al., 2022). The foundation of ConvNeXt came from the idea to take a hybrid approach to designing a convolutional neural network by modernizing “a standard ResNet toward the design of a vision Transformer, and discover several key components that contribute to the performance difference along the way.” Although ConvNeXt was not designed for hand gestures specifically, it can be trained to be used for hand recognition. Training this convolutional neural network to be used for hand gesture recognition leads to great performance, with ConvNeXt achieving “87.8% ImageNet top-1 accuracy and outperforming Swin Transformers on COCO detection ADE20K segmentation” while being simple and efficient like other convolutional neural networks.

Another example of a SOTA method for hand tracking is MeGaTrack, Meta’s robust system for identifying hand gestures and overlaying hand models for use in their VR headsets. The method uses key-point detection as well along with tracking history to reduce jitter in the model’s hand estimations. As drawing symbols and higher-detailed objects requires precision, such qualities would be highly beneficial to our model. Furthermore, Meta’s method uses a technique called “detection-by-tracking” which optimizes real-time computation for use in less powerful hardware such as VR headsets. Implementation of this technique in our model would help our model fulfill its role as a background process that can be used on top of another application like Zoom without taking up too much of the computer’s resources. One disadvantage of this method is that it is designed for a more robust task (3D hand tracking) on more specialized hardware. However, if we are able to scale down some of the methods they used to work for our specific task with basic computers and webcams, it could be extremely valuable in reaching our goals for this project.

Throughout the process of developing the project we will use several algorithms to compare performance of these SOTA methods. We will also do human trials using these SOTA methods to determine the feasibility with our own machines. With the information we gathered from these tests, we will determine the best SOTA method that best suits our project. First, we will research existing literature to see what has the best effectiveness for our specific problem. We will then eliminate any methods that don’t work for our problem domain.

Next, once in the development phase, we will explore various numerical methods of assessing our results by analyzing the difference between predicted and actual locations of hand key points. Some possible approaches are the Euclidean distance algorithm, the Mean Squared Error, and the Percentage of Correct Keypoints. All of these algorithms are standard statistical methods of measuring accuracy of predictions in a dimensional space. For assessing hand detection performance, we can treat it as a classification problem. We can assign a positive if the model has a prediction confidence of >0.5 and assign a correct result if hand prediction falls within certain distance parameters of the actual hand location (Han et al. 2020). The model’s performance can then be assessed using precision and recall metrics. We will try all of these methods and determine which can best be used to refine and analyze our model’s results.

In the next section, we will discuss how each of our SOTA methods works in depth, including a workflow diagram and each method’s advantages and disadvantages. After this, we will then provide which SOTA method we believe is the best one for our application and why, including preliminary testing results from each of our five potential SOTA methods stated above. We will then conclude this paper with a brief description of some lessons we have learned along the way as well as our timeline for the remainder of our project.

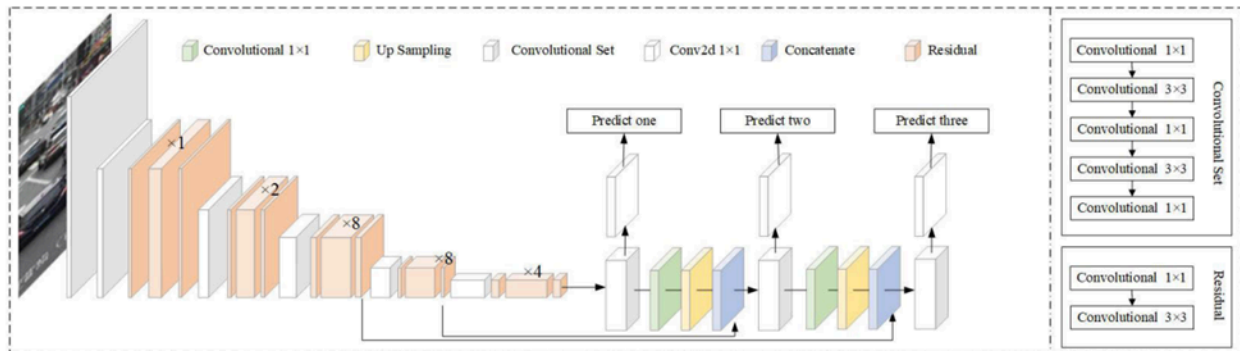
SOTA Literature Review

YOLOv3 and DarkNet-53

YOLOv3 is a machine learning algorithm that can detect objects within still images or during a real-time camera feed. YOLOv3 is capable of doing this by using a convolutional neural network (CNN) called DarkNet-53. The architecture of DarkNet-53 is described in *DarkNet-53: A Powerful Backbone for Object Detection* by Vipas.AI. DarkNet-53 is composed of 53 convolutional layers. There are 1x1 and 3x3 convolutional layers which “are arranged in a pattern that maximizes feature extraction while minimizing computational complexity” (Vipas.AI, 2024). The 1x1 convolutional layer reduces the size of the image. It is then inputted into the 3x3 convolutional layer to determine features. This pattern minimizes computational complexity while delivering high performance. DarkNet-53’s other focal point is its use of residual connections. This allows it to “train deeper models by mitigating the vanishing gradient problem” (Vipas.AI, 2024). This empowers the model to have a faster training time and have stronger object detection ability over its predecessors. (Vipas.AI, 2024)

There are a couple advantages and disadvantages to using the YoOLOv3 and DarkNet-53 approach. One advantage is the ability to train a high-accuracy model with images that do not require preprocessing, filtering, or enhancements (Mujahid et al., 2021). Another advantage is training a deep learning model relatively fast while offering strong object detection abilities (Vipas.AI, 2024). One disadvantage of using this approach is that there are newer versions of YOLO that may be more accurate or faster, however, they may not be necessary for our application. We also found a couple more disadvantages with this approach which stemmed from the computers we are using, but this will be discussed more in the *Preliminary Experimental Results and Discussions* section of this paper.

Figure 1
YOLOv3 Algorithm Workflow Structure



(Taken from *Mini-YOLOv3: Real-Time Object Detector for Embedded Applications*, 2019)

Figure 2

DarkNet-53's architecture

	Type	Filters	Size	Output
1x	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
2x	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	256	3 × 3 / 2	16 × 16
8x	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			16 × 16
	Convolutional	512	3 × 3 / 2	8 × 8
8x	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			8 × 8
	Convolutional	1024	3 × 3 / 2	4 × 4
4x	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			4 × 4
	Avgpool		Global	
	Connected		1000	
	Softmax			

(Taken from *DarkNet-53: A Powerful Backbone for Object Detection*, 2024)**EfficientNet**

In the previous section we had discussed using Faster R-CNN to handle locating the hands of the user and predicting what gesture the user was making. However, this decision was ultimately decided as impractical. The existing trained models for Faster R-CNN did not have a setting designed to detect hands or types of hands, and would have had to be trained mostly from scratch, which would have taken a lot of time and processing power. Additionally, the data would have required bounding boxes being drawn on, something that would also have taken time to set up. Finally, after performing early testing, faster R-CNN is not fast enough for a real-time video analysis. Since analysis occurs each frame, the output will either be delayed, or lag the video frames as it waits for output.

Because of these failures, we decided to test EfficientNet instead. EfficientNet is a machine learning algorithm designed for image detection. It is based around the architecture of a convolutional neural network. However, while traditional neural networks achieve effective results by adding more layers or filters, EfficientNet uses compound scaling to balance out these two features (Duong 2020). This means that while regular CNNs can scale only on width and depth, EfficientNet can also scale on the image resolution. In practice, this means that it can grow much larger than a traditional CNN and still achieve a similar level of effectiveness. EfficientNet is an image classifier, so it creates its output based on images that are put into certain categories. This means, unlike Faster R-CNN, it can be trained on simple unedited images that are pre-sorted into categories.

The pros of this model are that it is fast enough, and fairly effective. After performing tests of the model, the analysis can keep up with a real-time video feed, unlike Faster R-CNN. Additionally, the results were mostly effective, which will be more explored in a later section. However, this model is held back by its limitations in knowledge. While it can mostly determine what hand motion the figure is making, it cannot determine where the hand is, as it is only a classifier. For any drawing app this information would also be important, so EfficientNet could only exist as an assistant to another algorithm to create our goal of drawing capability.

Figure 6

The outline of the proposed network architecture for Inception V3.

type	patch size/stride or remarks	input size
conv	3×3/2	299×299×3
conv	3×3/1	149×149×32
conv padded	3×3/1	147×147×32
pool	3×3/2	147×147×64
conv	3×3/1	73×73×64
conv	3×3/2	71×71×80
conv	3×3/1	35×35×192
3×Inception	As in figure 5	35×35×288
5×Inception	As in figure 6	17×17×768
2×Inception	As in figure 7	8×8×1280
pool	8 × 8	8 × 8 × 2048
linear	logits	1 × 1 × 2048
softmax	classifier	1 × 1 × 1000

Note. From “Rethinking the Inception Architecture for Computer Vision” by Christian Szegedy et al.

<https://doi.org/10.48550/arxiv.1512.00567>

Another innovative feature of the Inception V3 model is the use of auxiliary classifiers. These act as intermediate layers during training to help guide the optimization process. These networks serve as regularizers by providing some additional analysis. In the Figure above these networks are within the *block x* that are scattered throughout. This innovation allows the architecture to be deeper without the risk of overfitting, thereby enhancing training stability and generalization to new data (Szegedy et al., 2015). In addition to these architectural adjustments, this model also includes techniques like label smoothing, which prevents the model from becoming overly confident in its predictions which improves its ability to be generalized to new data (Szegedy et al., 2015).

Ultimately, the overall design of Inception V3 improves efficiency allowing it to interpret high-resolution images and complex patterns without excessive computational demand. This is achieved through a modular design, where different types of convolutional filters are processed alongside one another and the outputs are concatenated to form a more accurate feature representation. Using these innovative optimizations, this model achieves great accuracy on benchmark datasets compared to other models, this is seen in Figure 7 below (Szegedy et al., 2015).

Figure 7

Single-model, multi-crop experimental results comparing the cumulative effects on the various contributing factors. Inception V3 compares its numbers with the best published single-model inference results on the ILSVRC classification benchmark.

Network	Crops Evaluated	Top-5 Error	Top-1 Error
GoogLeNet [20]	10	-	9.15%
GoogLeNet [20]	144	-	7.89%
VGG [18]	-	24.4%	6.8%
BN-Inception [7]	144	22%	5.82%
PReLU [6]	10	24.27%	7.38%
PReLU [6]	-	21.59%	5.71%
Inception-v3	12	19.47%	4.48%
Inception-v3	144	18.77%	4.2%

Note. From “Rethinking the Inception Architecture for Computer Vision” by Christian Szegedy et al.

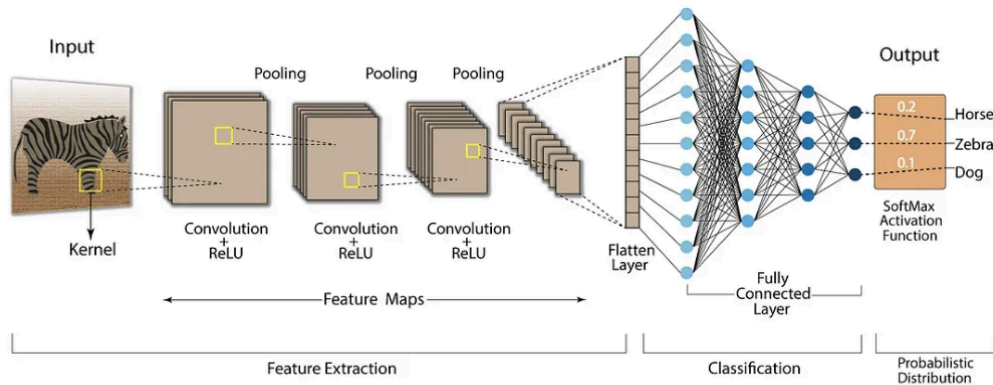
<https://doi.org/10.48550/arxiv.1512.00567>

ConvNeXt

In previous iterations of this paper, we had discussed using TriHorn-Net, which is a model that is used to improve accuracy in depth-based 3D hand pose estimation. This estimation is accomplished through “decomposing the 3D hand pose estimation into the estimation of 2D joint locations in the depth image space (UC), and the estimation of their corresponding depths aided by two complimentary attention maps,” (Rezai et al., 2023). TriHorn-Net also utilizes a method called PixDropout to assist in this estimation. While TriHorn-Net is incredibly impressive, we do not possess the technological resources, such as depth cameras, to utilize TriHorn-Net to our standard. This limitation led to our team deciding to use ConvNeXt.

ConvNeXt, as discussed in the *Introduction* section, ConvNeXt is a type of convolutional network model that takes a hybrid approach to designing a convolutional neural network by using approaches similar to Transformers. Figure 8 displays a diagram of the structure of a convolutional neural network, where parts of the image are pooled, flattened, then classified, where these classifications lead to the output of the network, which leads to the estimation of what the subject of the image is based on the part of the image. ConvNeXt accomplishes this through adding in an extra layer called global average pooling (GAP), where outputs of the convolutional layers are flattened into a single vector, which is taken from the convolutional neural network structure (Erdogan, 2023). The GAP is useful in that it creates a fixed size for feature maps, with a 5x5 feature map resulting in a 1x1 feature map in the GAP layer. As seen in Figure 9, the “block” structure of ConvNeXt, another feature of ConvNeXt is layer normalization. Layer normalization (which is labeled LN in Figure 9) is used to normalize the outputs of a network layer. This is done through calculating the mean and standard deviation of each map for each sample, which is then normalized by the previously mentioned mean and standard deviation. These are used in many Transformers, however, there are fewer per “block”, as seen in Figure 10, which displays a Swin Transformer “block,” (Liu et al., 2022). Finally, as seen in Figure 9 as well, is the Gaussian Error Linear Units activation function (abbreviated as GELU in Figure 9). This provides an output for values larger than zero and a non-linear output for negative values. As seen in Figure 10 as well, this is commonly used in other Transformer systems. The results in Figure 11 showcase how ConvNeXt competes rather well with models such as Swin Transformers and EfficientNet, leading to accuracy no lower than 82.1%, which is quite impressive.

Figure 8
Convolutional Neural Network Diagram

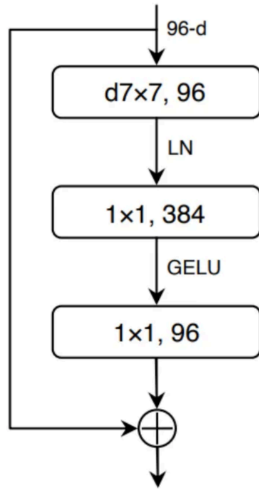


Note: From “ConvNeXt – Next Generation of Convolutional Networks” by Erdogan.

<https://medium.com/@atakanerdogan305/convnext-next-generation-of-convolutional-networks-325607a08c46>

Figure 9

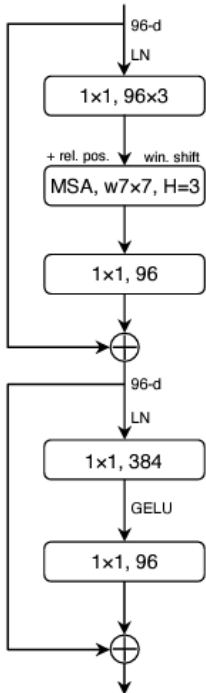
ConvNeXt Block Structure



*Note: From “A ConvNet for the 2020s by Liu et al.
<https://arxiv.org/abs/2201.03545>*

Figure 10

Swin Transformer Block Structure



*Note: From “A ConvNet for the 2020s by Liu et al.
<https://arxiv.org/abs/2201.03545>*

Figure 11
ConvNeXt Block Structure

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
• RegNetY-16G [54]	224 ²	84M	16.0G	334.7	82.9
• EffNet-B7 [71]	600 ²	66M	37.0G	55.1	84.3
• EffNetV2-L [72]	480 ²	120M	53.0G	83.7	85.7
◦ DeiT-S [73]	224 ²	22M	4.6G	978.5	79.8
◦ DeiT-B [73]	224 ²	87M	17.6G	302.1	81.8
◦ Swin-T	224 ²	28M	4.5G	757.9	81.3
• ConvNeXt-T	224 ²	29M	4.5G	774.7	82.1
◦ Swin-S	224 ²	50M	8.7G	436.7	83.0
• ConvNeXt-S	224 ²	50M	8.7G	447.1	83.1
◦ Swin-B	224 ²	88M	15.4G	286.6	83.5
• ConvNeXt-B	224 ²	89M	15.4G	292.1	83.8
◦ Swin-B	384 ²	88M	47.1G	85.1	84.5
• ConvNeXt-B	384 ²	89M	45.0G	95.7	85.1
• ConvNeXt-L	224 ²	198M	34.4G	146.8	84.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	85.5
ImageNet-22K pre-trained models					
• R-101x3 [39]	384 ²	388M	204.6G	-	84.4
• R-152x4 [39]	480 ²	937M	840.5G	-	85.4
• EffNetV2-L [72]	480 ²	120M	53.0G	83.7	86.8
• EffNetV2-XL [72]	480 ²	208M	94.0G	56.5	87.3
◦ ViT-B/16 (▣) [67]	384 ²	87M	55.5G	93.1	85.4
◦ ViT-L/16 (▣) [67]	384 ²	305M	191.1G	28.5	86.8
• ConvNeXt-T	224 ²	29M	4.5G	774.7	82.9
• ConvNeXt-T	384 ²	29M	13.1G	282.8	84.1
• ConvNeXt-S	224 ²	50M	8.7G	447.1	84.6
• ConvNeXt-S	384 ²	50M	25.5G	163.5	85.8
◦ Swin-B	224 ²	88M	15.4G	286.6	85.2
• ConvNeXt-B	224 ²	89M	15.4G	292.1	85.8
◦ Swin-B	384 ²	88M	47.0G	85.1	86.4
• ConvNeXt-B	384 ²	89M	45.1G	95.7	86.8
◦ Swin-L	224 ²	197M	34.5G	145.0	86.3
• ConvNeXt-L	224 ²	198M	34.4G	146.8	86.6
◦ Swin-L	384 ²	197M	103.9G	46.0	87.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	87.5
• ConvNeXt-XL	224 ²	350M	60.9G	89.3	87.0
• ConvNeXt-XL	384 ²	350M	179.0G	30.2	87.8

Note: From “A ConvNet for the 2020s by Liu et al.
<https://arxiv.org/abs/2201.03545>

MediaPipe Hands

In the previous literature review we discussed Meta’s MeGATrack as a potential method to implement in our solution, but unlike some of the other models, MeGATrack is not open source and depends on specialized hardware, so we could not test or use the model directly. However we found a viable replacement in Google’s open source model MediaPipe Hands. MediaPipe Hands’ workflow closely resembles that of MeGATrack as it also uses a pipeline of two different models to efficiently and accurately determine key hand landmarks from an input image. The first model is a palm detector that extracts only the part of the image that contains a detected hand. The second model then receives the simplified output of the previous one, makes the final landmark predictions, and outputs the results to the user. The results from the previous prediction are then used to determine hand location for the next prediction, greatly cutting down on the need to use the first hand-detection model.

The main advantages of the MediaPipe Hands pipeline are its speed and computational efficiency. The use of two different specialized models to break down the problem greatly reduces overhead from data cleaning and preparation before making the final prediction. The feedback architecture of the pipeline helps reduce redundancy in calculations and allows the pipeline to devote more processing to the landmark predictions. The hand detecting model itself uses a single-shot detecting technique that is optimized to run on mobile and lower power GPUs, further reducing computation times. These optimizations all allow the model to make complex inferences from input data fast enough to generate accurate predictions in real time on a live video feed. The only downside to Google’s MediaPipe Hands is that it is not an out-of-box gesture detecting solution. We will have to use raw data provided by the model to make inferences about gestures, which leaves room for accuracy issues. However, Google does provide a suggested method for interpreting the models output for gesture recognition which should minimize inaccuracies.

Figure 12

High-Level Map of Inputs and Outputs in the MediaPipe Hands Pipeline

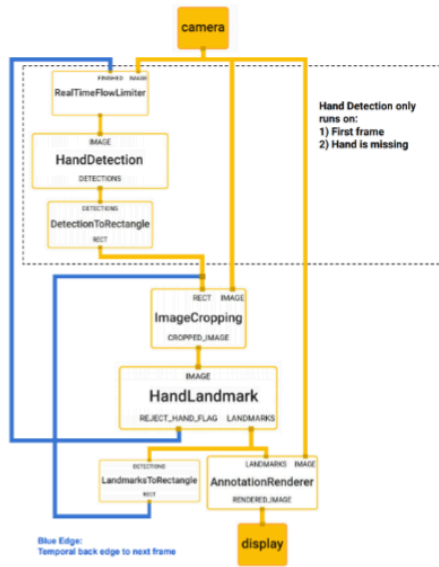
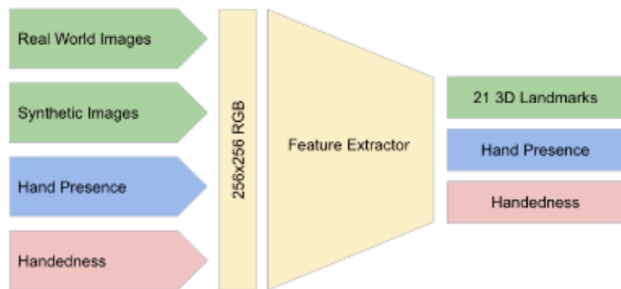


Figure 13

Input/Output Architecture For the Hand Landmark Model



Proposed High-level Solution and Process

EfficientNet

In order to create an implementation of EfficientNet, we used the torchvision library to get access to pre-trained models of efficientNet to begin with. Then, we used transfer learning, which involves using existing parameters in a pretrained model as a place to begin for new training. We created a dataset of several hundred images of hands in various poses, classified them, and then transformed them into a form that would be compatible with the starting efficientNet model.

After everything was set up, we began training the model. This involves two phases; a training phase and a validation phase. In the training phase, the model processes the inputs, calculates output predictions, and judges how accurate it currently is based on the image labels. Then it updates the model parameters based on what changes will most effectively minimize the loss. In the validation phase, the algorithm iterates over all of the choices it has previously made and tests them; after confirming and updating, this loop proceeds anew until the number of iterations have been satisfied. (Appendix 1)

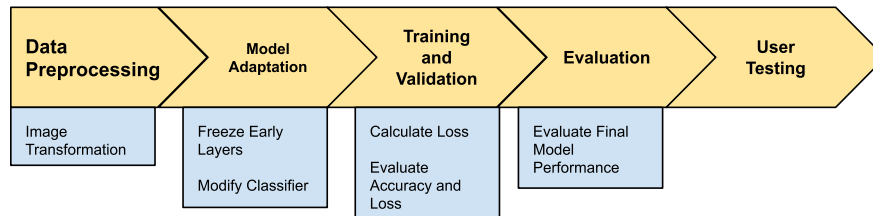
Finally, the model is tested and run. Testing involves analyzing the completed model's output based on a wholly new set of pictures, and rating its performance on our predetermined metrics. Running it involves taking a video stream, analyzing the models' output on each frame, and printing the predicted classifier to the screen to compare. This step of human testing is crucial; while the model may appear to succeed due to the metrics, if the dataset is too small it could miss many edge cases encountered during practical testing.

Inception V3

A proposed solution that uses the Inception V3 architecture is discussed in this section. The primary goal is to fine-tune Inception V3 to enhance its performance on challenging backgrounds, reduce computational overhead and improve accuracy for the task of drawing, stopping and erasing. The workflow for this solution approach involves some key steps: data preprocessing, model adaptation (transfer learning), training, validation and evaluation. Figure 14 below illustrates the workflow of the system, making sure to highlight the interactions between different components of the pipeline.

Figure 14

Workflow of the proposed solution using Inception V3



First, there is data preprocessing, as seen in Appendix II the first portion of the code is dedicated to resizing the images to 299x299 pixels, normalization to standardize color channels and then convert them to tensors. This ensures that the input data is compatible with the model architecture and optimizes the training process on our dataset (PyTorch, n.d).

Next, a pretrained Inception V3 model is used, allowing the network to see features that have been learned from large datasets such as ImageNet, which the model was trained on prior to use in this project. Transfer Learning is then applied, this is a machine learning technique where knowledge gained through one dataset (in this case ImageNet) is used to improve model performance on another related task (AI Whiteboard gesture recognition). In

the code this is achieved by freezing the earlier layers to retain pre-trained feature recognition while fine tuning the last classification layers to adapt to the specific dataset.

In order to train the model we use a custom function that handles the structure of this model, mainly the use of its auxiliary classifiers during training. The loss is calculated as a weighted sum of the main output loss and auxiliary output loss. Post training and validation, the model is evaluated on a Test set to determine its accuracy, precision and recall. Metrics like these are important because they help understanding the performance of the model across different classes, especially when dealing with imbalanced datasets for a classification task.

The next step often considered the most important is to test the model for the task at hand which is an AI Whiteboard, in-order to do this we use a live feed that loads the pretrained model and uses frames from the live feed to determine what gesture it is. The program then interprets this hand-gesture and displays the gesture on the screen so that the user can see what this gesture is. If this was used for the final iteration, these different classifications would then translate to actual “draw”, “erase” and “stop” actions.

ConvNeXt

In creating the ConvNeXt, we utilized the Tensorflow and PyTorch Python libraries. Within Tensorflow, the Keras portion of the libraries provided aspects of the project such as models, layers, callbacks, and more. As previously mentioned, a dataset of hundreds of hand images showcasing gestures used for stop, draw, and erase. The dataset was split into two folders, dataset and train, with the dataset folders having all hand gesture images representing the subfolder they are within, and the test folders having specially selected images representing the subfolder they are within. After this process, ImageDataGenerators were created for both training and testing.

To create the model, a ConvNeXt model from Tensorflow’s Keras library was utilized. The model was built, compiled, then trained with validation data. In order to build the model, Tensorflow’s Keras library’s ConvNeXtBase implementation of the ConvNeXt model was utilized with a pretrained imagenet model utilized for the weight. Training the model is done after the compilation of the model, and consists of utilizing the train data and validation data to train via passing through the ConvNeXt model layers, where operations such as convolutions and normalization are carried out to help the model recognize hand gestures. For evaluation, the same operations from the previous step are carried out on the test data. After the evaluation, prediction using the model is conducted on the test data as well utilizing past layer operations.

MediaPipe Hands

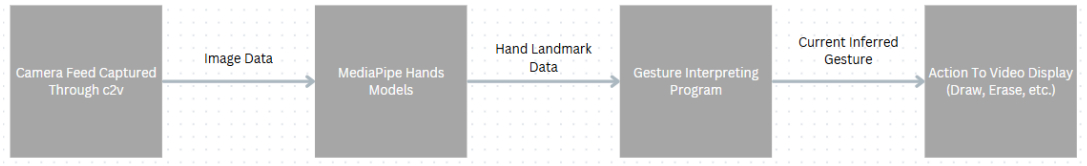
The MediaPipe Hands model pipeline offers a promising solution to the problem of an AI whiteboard. The optimized nature of the pipeline would allow for accurate and fast hand landmark predictions even in the constrained environment of a Zoom or Teams plugin. The model returns raw data to be interpreted which might increase accuracy in recognizing gestures that have been rotated or deformed, allowing for more flexible and intuitive use of the application. Furthermore, the model inherently detects the placement of the tip of the index finger so the solution would be able to perform the draw action without any further predictions or calculations.

An implementation of a solution using MediaPipe Hands would require four key elements. 1) A camera feed to gather inputs and display outputs. This element can be implemented using the c2v library which has built-in capture and display capabilities as well as useful functions for interfacing with ML models. 2) An implementation of the MediaPipe hands pipeline to get hand landmark predictions. The model can be imported using the mediapipe library and implemented using the setup instructions in the open-source documentation. 3) A method for reliably interpreting landmark data as gestures. The technique outlined by Google involves calculating the angle for each joint in a finger, summing them up, comparing them to threshold values to determine if a finger is curled, extended or neither. Using different patterns of extended and curled fingers one can then classify the overall gesture of the hand. 4) A method of displaying outputs to the video feed correctly. Once again the c2v library will be useful as it supports the drawing of small circles on images which can be placed at the detected coordinates of the index finger

at regular intervals to draw. The final product will allow the user to draw on the screen with their index finger and erase with a closed fist.

Figure 15

Proposed High-Level Implementation of MediaPipe Hands



Preliminary Experimental Results and Discussions

YOLOV3

While experimenting and evaluating the SOTA method YOLOv3 and DarkNet-53, we ran an evaluation script to generate metrics for the model's precision, recall, and accuracy. Our evaluation script concluded that the pretrained model we found to detect hand gestures had a precision metric of 0.1, a recall metric of 0.63, and an accuracy metric of 0.35. We also ran the real time video feed object detection feature of the algorithm. Sadly, at least on the computers we were evaluating the YOLOv3 algorithm on, it did not work very well. The video feed was very laggy and often missed assigned hand gestures. It also seemed to only work if your hand was close to the camera, which would not be optimal for our application. However, we are unsure if this is caused by the YOLOv3 algorithm itself or if the data it was trained with was solely close up hand images. Due to these reasons, we concluded that the evaluation results and the real time gesture recognition was not accurate enough. We also determined that the computers we have access to do not have the necessary computing power to run this algorithm optimally.

EfficientNet

In order to test EfficientNet, we created an evaluation script that used test data to find precision, recall, and accuracy. EfficientNet had much different responses than our other test models. With the same test data as the others, this model had nearly perfect accuracy. This is a sign that our testing data was not complete enough, and that we needed more pictures to catch edge cases and other unnoticed cases. In terms of physical testing, the model was mostly effective, but occasionally got confused when quickly switching between hand states. Ultimately we determined that it was not effective enough. Additionally, the model could not determine other information like hand location, meaning we decided not to use this model.

Inception V3

Now we discuss the results for the Inception V3 Model that was fine-tuned for our task. This model was trained for 10 epochs, with the performance of each epoch evaluated across several metrics such as training loss, validation loss and validation accuracy. Furthermore, in order to assess its overall generalization, we tested using test accuracy, precision and recall. These metrics allow the user to understand how well the model learns from the training data, adapts to unseen data and generalizes to completely new test samples. The figure below showcases the results of the tests that are later discussed.

Figure 16

Results from training the Inception V3 Model

```
Epoch 1/10, Train Loss: 1.42311554774642, Val Loss: 0.7842787504196167, Accuracy: 72.95%
Epoch 2/10, Train Loss: 1.213822290301323, Val Loss: 0.6443012095987797, Accuracy: 77.05%
Epoch 3/10, Train Loss: 1.1553745828568935, Val Loss: 0.5853457078337669, Accuracy: 74.59%
Epoch 4/10, Train Loss: 1.059415653347969, Val Loss: 0.5137630105018616, Accuracy: 84.02%
Epoch 5/10, Train Loss: 1.0308972746133804, Val Loss: 0.45932959765195847, Accuracy: 88.52%
Epoch 6/10, Train Loss: 1.0062577705830336, Val Loss: 0.4605226628482342, Accuracy: 81.97%
Epoch 7/10, Train Loss: 0.9404780603945255, Val Loss: 0.45785954035818577, Accuracy: 78.69%
Epoch 8/10, Train Loss: 0.9577113837003708, Val Loss: 0.4100647270679474, Accuracy: 85.66%
Epoch 9/10, Train Loss: 0.9198920056223869, Val Loss: 0.4102704506367445, Accuracy: 86.89%
Epoch 10/10, Train Loss: 0.9188369419425726, Val Loss: 0.36745929904282093, Accuracy: 89.34%

Test Accuracy: 83.00%
Precision: 0.85
Recall: 0.81
```

During the training and validation phase, the model shows that there is an improvement across epochs. As seen in the results the Training Loss goes from 1.42 in the first epoch to 0.918 in the tenth. This indicates that the difference between the predicted and actual is low. However, this does not mean that the model will perform well on new data. Also, the validation loss decreased from 0.784 to 0.367, this indicates that the model was learning and minimizing errors.

For the testing process, the model achieved an overall accuracy of 83%, a precision score of 0.85 and a recall score of 0.81. It is important to recall that precision measures the proportion of true positive predictions among positive predictions, and recall measures the proportion of true positive predictions among all positive instances (google, 2024). The high precision score indicates that the model is trustworthy in its positive classifications while the recall score indicates that the model is better at identifying most of the draw, stop and erase classifications in the dataset.

While the results from training and testing the model were promising, we encountered challenges when implementing the real-time hand gesture recognition via a live feed. When using a webcam, the live feed exhibited jittery behavior and some hand gestures were incorrectly identified. This issue can be seen in the figure below. These issues can be attributed to a variety of factors, such as variations in lighting, background noise and the computational load that is required to process this model in real time. When we consider the use case for this model it becomes clear that this would not be a good option as it is jittery and does not recognize gestures appropriately, therefore it would not be the most user-friendly approach.

Figure 17

Incorrect Classification of hand-gestures from a live feed.



Note. The image on the left represents a “stop” gesture, however; the model interprets it as a “draw” gesture. The image on the right represents a “draw” gesture but the model interprets it as a “stop” gesture.

ConvNeXt

Once the model was finished running its operations, metrics for ConvNeXt were output. These metrics displayed that using ConvNeXt resulted in a precision average of around .44 for the draw, erase, and stop gestures. The accuracy was similar, with a .44 average. Running the video feed was very laggy, as ConvNeXt can be quite computationally expensive. Running the program also seemed to only work when close to the camera, but that could also be a result of the cameras we have access to in order to carry out these tests. Due to the low accuracy and poor performance when running with the camera, we came to the decision that ConvNeXt was not the best accurate enough and not compatible with the technology we have access to, a similar result to the results we received when working with YOLOv3.

MediaPipe Hands

A method was implemented to interpret the raw data outputted by MediaPipe Hands and it was tested against the test split of the data. The resulting metrics were as follows: Accuracy: 0.34, Precision: 0.75, Recall: 0.30. These results show poor preliminary performance of the model for predicting the given gestures, but this likely reflects more on the method of interpreting the output of the model than the model's accuracy itself. A preliminary method was used for this phase to test out the model and these results are promising enough to pursue a more robust method of interpretation for future iterations. Furthermore, despite poor performance in the numerical tests, the model has shown promising performance in live tests with low jitter, and acceptable prediction fidelity, so we will continue to explore the possibility of a solution using MediaPipe hands.

Final Results and Selected Model

SOTA Method	Precision	Accuracy	Recall	Live Performance
YOLOV3	0.1	0.63	0.35	Mid
Inception V3	.83	.85	.81	Mid
ConvNeXt	.44	.44	.44	Mid
Mediapipe Hands	.34	0.75	.3	Very good

*EfficientNet removed from this table due to having outlier values

For our final product we decided to implement Google's MediaPipe Hands pipeline for its live performance and high relevance to our task. Though in the numerical benchmarking MediaPipe had poor performance compared to models like EfficientNet, it outperformed all other models in live testing. Unlike the other models which suffered issues in consistency and framerate, MediaPipe had relatively smooth and jitter-free gesture predictions. After tuning our algorithm for interpreting the model's output and implementing a confidence threshold, the model was able to easily draw clean lines even with a relatively busy background.

Furthermore, the low metrics we observed in recall and accuracy could be due to a misalignment between our test dataset and our gesture interpretation methods. There were some specific conditions that we implemented for intuitive use of the end product that didn't line up with how gestures were labeled in the test set, causing poor metrics.

On top of everything else, one of the greatest benefits of our selected model was the raw location information it provided. Though this information required an interpretation algorithm to detect gestures, it provided us with the location of the user's fingertip for drawing. If we used any of the other models for gesture detection we would need an entirely separate model to extract the location information. Google's MediaPipe Hands gave us all the information we needed out of the box.

Lessons Learned

Through the work completed at this point in the project, many lessons have been learned. One of these lessons is that computational and other hardware resource limitations can severely hinder a project. This can be seen with the change from TriHorn-Net to ConvNeXt due to a lack of depth cameras being accessible for this project. Device hardware limitations have also been an issue, with models taking quite a long time to train due to less

powerful GPUs and CPUs we have access to. Another aspect we learned is that artificial intelligence can be quite difficult to work with and leaving extra time for debugging and fixing issues is very important.

Another challenge that we encountered was that implementing a live feed for hand gesture recognition introduced additional complexities, such as jittery performance and misclassification of gestures. These issues stemmed from both computational limitations as well as inherent limitations of the model. To address these challenges in the future without final iteration we plan on using methods to smooth and stabilize live video input. While our systems may be a constraint, this highlighted the need for optimizing the current system to achieve smoother real-time outputs.

This iteration also had us learn more about the value of experimenting with different approaches and iterating on the solution. From utilizing transfer learning and measuring the training and validation to evaluating the final model. We learned about the criticality of continuously testing, measuring and adjusting to achieve the best results for our AI Whiteboard.

Overall, we believe that this project has been a learning experience. We have acquired practical knowledge in the field of computer vision and deep learning as well as taking the opportunity to work with widely used libraries like TensorFlow and PyTorch. Additionally, we learned about managing performance metrics and dealing with real-world challenges, such as computational constraints and real-time processing which will be beneficial for future projects in machine learning and artificial intelligence.

Future Work

The main focus of future work should be focussed towards incorporating our system into virtual meeting services such as Zoom and Google Meet. Since this is the target use of our system, it should be the primary focus to add it as a Zoom or Google Meet tool. However, before this can happen, we believe some improvements can be made to the tool before incorporating it. One main feature we believe would be beneficial to our software is adding a color wheel. This will allow the user to choose between a variety of colors to use while drawing on the screen. Another feature we believe would be beneficial is adding jitter compensation. This will enable the user to make cleaner drawings with smoother and straighter lines. Finally, we would also like more drawing features incorporated such as shape drawing. For example, this will allow the user to draw a circle, and the system will automatically convert this into a perfect circle, similar to what is currently done on certain note taking applications.

Conclusions

As discussed thoroughly throughout this document, AI Whiteboard has been given much thought in order to create the best project possible within our limited resources. As seen in previous sections of this document, different models were tested in order to choose a model that would best suit our needs and fit within our limitations, eventually arriving at the decision to utilize MediaPipe Hands due to it being the method providing the best performance. Throughout the research, training, testing, and implementation processes, we learned many lessons about working with artificial intelligence, such as working with technical limitations. We believe this project has incredible potential and should not be abandoned. Thus, we have many more ideas to implement into AI Whiteboard, such as meeting service plugin support, as previously mentioned. Developing AI Whiteboard was an enjoyable process and we are very proud of the final result.

References

Anusha, S. B., G. H. Samyama Gunjal, & Manjushree, N. S. (2023). Static Hand Gesture Prediction Using Inception

V3. Cognitive Science and Technology, 121–133. https://doi.org/10.1007/978-981-19-2358-6_13

- Duong, L. T., Nguyen, P. T., Di Sipio, C., & Di Ruscio, D. (2020). Automated fruit recognition using EfficientNet and MixNet. *Computers and Electronics in Agriculture*, 171, 105326.
<https://doi.org/10.1016/j.compag.2020.105326>
- Erdogan, A. (2023). ConvNeXt – Next Generation of Convolutional Networks. Medium.
<https://medium.com/@atakanerdogan305/convnext-next-generation-of-convolutional-networks-325607a08c46>
- google. (2024). Classification: Accuracy, recall, precision, and related metrics. Google for Developers.
<https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>
- Hamilton, I., & Beagle, V. (2023, May 24). *The Rise Of Online Learning In The U.S. – Forbes Advisor*.
Www.forbes.com. <https://www.forbes.com/advisor/education/online-colleges/online-learning-stats/>
- Han, S., Liu, B., Cabezas, R., Twigg, C. D., Zhang, P., & Petkau, J. (2020, August 17). Megatrack: Monochrome egocentric articulated hand-tracking for virtual reality - meta research. Meta Research.
<https://research.facebook.com/publications/megatrack-monochrome-egocentric-articulated-hand-tracking-for-virtual-reality/>
- Karsh, B., Laskar, R. H., & Karsh, R. K. (2023). mIV3Net: modified inception V3 network for hand gesture recognition. *Multimedia Tools and Applications*, 83(4), 10587–10613.
<https://doi.org/10.1007/s11042-023-15865-1>
- L. Yang, Q. Song, Z. Wang, M. Hu and C. Liu, "Hier R-CNN: Instance-Level Human Parts Detection and A New Benchmark," in *IEEE Transactions on Image Processing*, vol. 30, pp. 39-54, 2021, doi: 10.1109/TIP.2020.3029901.
- Liu, Z., Mao, H., Wu, C., Feichtenhofer, C., Darrell, T., Xie, S., Facebook AI Research, UC Berkeley. (2022). A ConvNet for the 2020s. ArXiv (Cornell University). <https://arxiv.org/abs/2201.03545>
- Mujahid, A., Awan, M. J., Yasin, A., Mohammed, M. A., Damaševičius, R., Maskeliūnas, R., & Abdulkareem, K. H. (2021). Real-Time Hand Gesture Recognition Based on Deep Learning YOLOv3 Model. *Applied Sciences*, 11(9), 4164. <https://doi.org/10.3390/app11094164>
- Mao, Q.-C., Sun, H.-M., Liu, Y.-B., & Jia, R.-S. (2019). Mini-YOLOv3: Real-Time Object Detector for Embedded Applications. *IEEE Access*, 7, 133529–133538. <https://doi.org/10.1109/access.2019.2941547>

Nguyen, C. T. (2021, March 11). Perspective | Zoom classes felt like teaching into a void — until I told my students why. *Washington Post*.

https://www.washingtonpost.com/outlook/zoom-classes-felt-like-teaching-into-a-void--until-i-told-my-students-why/2021/03/11/024afb2c-81e5-11eb-81db-b02f0398f49a_story.html

PyTorch. (n.d.). [Www.pytorch.org. https://pytorch.org/hub/pytorch_vision_inception_v3/](https://pytorch.org/hub/pytorch_vision_inception_v3/)

Rezaei, M., Rastgoo, R., & Athitsos, V. (2023). TriHorn-net: a model for accurate depth-based 3D hand pose estimation. *Expert Systems with Applications*, 223, 119922.

<https://www.sciencedirect.com/science/article/abs/pii/S0957417423004232>

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). Rethinking the Inception Architecture for Computer Vision. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1512.00567>

Tanveer, Shahroz. (2020). GitHub - ShahrozTanveer/Hand-Gestures-Recognition: Hand Gestures Recognition with YOLOv3. GitHub. <https://github.com/ShahrozTanveer/Hand-Gestures-Recognition>

Vipas.AI. (2024, August 8). DarkNet-53: A Powerful Backbone for Object Detection. Medium.

<https://medium.com/@vipas.ai/darknet-53-a-powerful-backbone-for-object-detection-40471eca5349>

Appendixes

Appendix I

```
# Training function
def train_model(model, criterion, optimizer, train_loader, val_loader, epochs=10):
    for epoch in range(epochs):
        model.train() # Set the model to training mode
        running_loss = 0.0

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad() # Reset gradients to zero

            outputs = model(inputs) # Forward pass
            loss = criterion(outputs, labels) # Calculate loss
            loss.backward() # Backward pass
            optimizer.step() # Update model parameters

            running_loss += loss.item() # Accumulate the loss

        # Validation loop
        model.eval() # Set the model to evaluation mode
        val_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad(): # Disable gradient computation for validation
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs) # Forward pass
                loss = criterion(outputs, labels) # Calculate loss
                val_loss += loss.item() # Accumulate validation loss

                _, predicted = torch.max(outputs.data, 1) # Get the predicted class
                total += labels.size(0) # Count total number of labels
                correct += (predicted == labels).sum().item() # Count correct predictions

        # Print epoch results
        print(f"Epoch {epoch+1}/{epochs}, "
              f"Train Loss: {running_loss/len(train_loader):.4f}, " # Average training loss
              f"Val Loss: {val_loss/len(val_loader):.4f}, " # Average validation loss
```

Code that executes for each training run of the EfficientNet algorithm

Appendix II

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, transforms, datasets
from torch.utils.data import DataLoader
from sklearn.metrics import precision_score, recall_score

# Define transformations (adjusting input size for Inception v3)
transform = transforms.Compose([
    transforms.Resize((299, 299)), # Inception v3 requires 299x299 input size
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Load datasets
train_data = datasets.ImageFolder("Train/", transform=transform)
val_data = datasets.ImageFolder("Val/", transform=transform)
test_data = datasets.ImageFolder("Test/", transform=transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

# Load pre-trained Inception v3 model (Transfer Learning)
model = models.inception_v3(pretrained=True)
for param in model.parameters():
    param.requires_grad = False

# Modify the model's classifier for the correct number of output classes
# Inception v3 has an `AuxLogits` layer, which is used only during training.
model.aux_logits = True
model.fc = nn.Linear(model.fc.in_features, len(train_data.classes))
model.AuxLogits.fc = nn.Linear(model.AuxLogits.fc.in_features,
                                len(train_data.classes))

# Define Loss Function and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)

# Training function
def train_model(m, c, o, t, v, epochs=10):
    # Determine if CUDA is available and set the device accordingly
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device) # Move the model to the appropriate device (GPU or CPU)

    for epoch in range(epochs):
        model.train() # Set the model to training mode
        running_loss = 0.0

        # Training loop
        for inputs, labels in train_loader:
```

```

        inputs, labels = inputs.to(device), labels.to(device) # Move data to
device
        optimizer.zero_grad() # Reset gradients to zero

        # Forward pass for Inception v3: requires handling auxiliary outputs
        outputs, aux_outputs = model(inputs)
        loss1 = criterion(outputs, labels)
        loss2 = criterion(aux_outputs, labels)
        loss = loss1 + 0.4 * loss2 # Combined loss

        loss.backward() # Backward pass (compute gradients)
        optimizer.step() # Update model parameters

        running_loss += loss.item() # Accumulate the loss

    # Validation loop
    model.eval() # Set the model to evaluation mode
    val_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad(): # Disable gradient computation for validation
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device) # Move data to
device
            outputs = model(inputs) # Forward pass
            loss = criterion(outputs, labels) # Calculate loss
            val_loss += loss.item() # Accumulate validation loss

            _, predicted = torch.max(outputs.data, 1) # Get the predicted class
            total += labels.size(0) # Count total number of labels
            correct += (predicted == labels).sum().item() # Count correct
predictions

    # Print epoch results
    print(f"Epoch {epoch+1}/{epochs}, "
          f"Train Loss: {running_loss/len(train_loader)}, " # Average training
loss
          f"Val Loss: {val_loss/len(val_loader)}, " # Average validation loss
          f"Accuracy: {100 * correct / total:.2f}%") # Validation accuracy

    # Save the model after training
    torch.save(model.state_dict(), "inception_v3_trained_model.pth")
    print("Model saved as inception_v3_trained_model.pth")

train_model(model, criterion, optimizer, train_loader, val_loader, epochs=10)

# Testing function
def model_evaluation(m, t):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    model.eval()

    all_labels = []
    all_predictions = []

```

```

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)

        all_labels.extend(labels.cpu().numpy())
        all_predictions.extend(predicted.cpu().numpy())

# Calculate accuracy
accuracy = 100 * (torch.tensor(all_predictions) ==
    torch.tensor(all_labels)).sum().item() / len(all_labels)
print(f"\nTest Accuracy: {accuracy:.2f}%")

# Calculate Precision and Recall (macro-averaged)
precision = precision_score(all_labels, all_predictions, average='macro')
recall = recall_score(all_labels, all_predictions, average='macro')

print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")

# Test the model
model_evaluation(model, test_loader)

```